

Instituto Tecnológico de Estudios Superiores de Monterrey
Campus Estado de México

Inteligencia artificial avanzada para la ciencia de datos (Gpo 601)

Profesores: **Jorge Adolfo Ramírez Uresti**
Elisabetta Crescio, Julio G. Arriaga Blumenkron, Israel Sanchez Miranda,
Víctor Adrián Sosa Hernández, Miguel González Mendoza, Alberto Michel Pérez Domínguez

**Módulo 2 Implementación de una técnica de aprendizaje máquina sin el uso de un
framework. (Portafolio Implementación)**

Árbol de Decisión

Emilio Páez De la Mora - A01801224

Dataset y separación

Para la implementación del árbol de decisión se utilizó el Online Shoppers Purchasing Intention Dataset (Sakar & Kastro, 2018, UCI Machine Learning Repository, licencia CC BY 4.0). Contiene 12,330 sesiones de un sitio de comercio electrónico, cada una perteneciente a un usuario a lo largo del año. La variable a predecir en este caso es Revenue es decir si la sesión terminó en compra

De las 18 columnas, solamente se utilizaron 10, las numéricas, además de Weekend y VisitorType, transformadas a valores binarios teniendo así un total de 12 variables. Las variables descartadas representan elementos categóricos sin un orden real, por lo que no tendrían significado o relevancia en un árbol que parte por umbrales numéricos. Es importante mencionar que el dataset no contiene valores faltantes.

Se separaron los datos en 80/20, es decir 9,864 registros de entrenamiento y 2,466 de prueba. Sumado a esto se utilizó una semilla fija (42) para garantizar la reproducibilidad de los resultados en implementaciones futuras.

Las clases se presentan fuertemente desbalanceadas. Un modelo que prediga siempre "no compra" obtendría 84.5% de accuracy sin haber aprendido nada. Ese es el piso contra el que hay que comparar cualquier resultado, y la razón por la que este reporte no se limita a reportar accuracy. La distribución de clases y los histogramas por variable están en resultados/distribucion_clases.png e histogramas_variables.png.

Implementación y decisiones de diseño

El algoritmo que se utilizó fue el de un árbol de decisión. Se utilizó Pandas únicamente para leer y manipular la tabla y seaborn/matplotlib para generar gráficas.

Los componentes que se implementaron manualmente fueron la entropía y el índice Gini como criterios de impureza, intercambiables mediante un parámetro, la ganancia de información con el promedio ponderado por el tamaño de cada subgrupo, la discretización de las variables continuas en cortes candidatos, la búsqueda del mejor corte entre todas las variables y todos los umbrales, la construcción recursiva del árbol, la predicción recorriendo el árbol desde la raíz hasta una hoja, la separación de los datos y el cálculo de la matriz de confusión junto con las cinco métricas reportadas.

En cuanto a las decisiones de diseño, la primera fue adaptar ID3 a variables continuas. En la presentación el algoritmo trabaja con atributos categóricos y abre una rama por cada valor posible, sin embargo las variables de este dataset son continuas, es decir que abrir una rama por valor produciría miles de ramas con un solo registro cada una. Por esta razón cada nodo realiza un corte binario, eligiendo un umbral y preguntando si el valor es menor a ese número, tal como lo hace CART.

La segunda decisión fue la de utilizar cortes candidatos por percentiles. Probar todos los puntos de corte posibles implicaría evaluar cerca de 9,864 umbrales por variable en cada nodo, por lo que en su lugar se calculan 32 candidatos repartidos por percentiles una sola vez al inicio del entrenamiento. La pérdida de precisión es mínima ya que los valores cercanos se comportan de manera similar, y el entrenamiento pasa de tardar minutos a menos de un segundo. Cabe mencionar que esta es la misma estrategia que utilizan implementaciones profesionales como LightGBM.

Finalmente se definieron cuatro criterios de paro. Un nodo se convierte en hoja si todos sus registros pertenecen a la misma clase, que es el caso base del ID3, si contiene menos de 20 registros, si alcanzó la profundidad máxima establecida, o si ningún corte logra reducir la impureza. Los últimos tres funcionan como controles contra el sobreajuste y su efecto se midió en la sección de experimentos.

Validación contra Play Tennis

Con el fin de comprobar que la implementación es correcta y no únicamente que aparenta funcionar, se reprodujo el ejercicio de Play Tennis visto en clase, cuyos valores fueron calculados manualmente en el material del curso. Esta validación se encuentra en el módulo `validacion_playtennis.py` y puede ejecutarse directamente con `python src/validacion_playtennis.py`. El archivo está organizado en bloques que alternan cada función con su comprobación, y termina importando las funciones de `arbol.py` para verificar que las que entrenan el modelo sobre el dataset real producen exactamente los mismos valores.

Calculo	Este archivo	arbol.py
-----	-----	-----
Entropy(S)	0.9403	0.9403
Gain(S, Outlook)	0.2467	0.2467
Gain(S, Temperature)	0.0292	0.0292
Gain(S, Humidity)	0.1518	0.1518
Gain(S, Wind)	0.0481	0.0481

Los cinco valores coinciden y la mayor ganancia corresponde a Outlook, que es efectivamente el atributo que ID3 selecciona como raíz en el ejercicio, lo cual valida tanto las fórmulas de impureza como la de ganancia de información.

Resultados: matriz de confusión y métricas

El modelo final utiliza entropía con una profundidad máxima de 5 y un mínimo de 20 muestras por nodo, dando como resultado un árbol de 51 nodos y 26 hojas. Los resultados obtenidos sobre los 2,466 registros de prueba, que el modelo nunca vio durante el entrenamiento, se presentan a continuación.

Matriz de confusion - modelo final

Real 0	1998	86
Real 1	153	229
	Predicho 0	Predicho 1

Figura 1. Matriz de confusión del modelo final sobre el conjunto de prueba.

```
Accuracy: 0.9031
Precision: 0.7270
Recall: 0.5995
Specificity: 0.9587
F1 Score: 0.6571
```

Accuracy por sí sola sería engañosa: el 0.9031 obtenido solo supera en 5.8 puntos al piso trivial de 0.845. Por eso se reportan además precision y recall, que separan los dos tipos de error, y F1, que los resume en un solo número. Specificity se incluye para hacer visible la asimetría entre clases.

En cuanto a la lectura de la matriz, el modelo acierta 1,998 no compras y 229 compras. Los 86 falsos positivos corresponden a sesiones sin compra marcadas como compra, mientras que los 153 falsos negativos son compras reales que se le escaparon, es decir el 40% de los 382 compradores del conjunto de prueba. El contraste entre una specificity de 0.9587 y un recall de 0.5995 es la firma directa del desbalance, ya que el modelo contó con 10,422 ejemplos de la clase mayoritaria y solamente 1,908 de la minoritaria.

Llevándolo al problema de negocio, si el modelo se utilizara para decidir a qué usuarios mostrar una promoción durante la sesión, cuatro de cada diez clientes con intención real de compra no la recibirían, mientras que un falso positivo únicamente representa mostrar una promoción de más. En este contexto conviene priorizar recall por encima de precision.

Experimentos: profundidad y criterios

Para el primer experimento se midió el efecto de la profundidad máxima. Se entrenó un árbol por cada profundidad midiendo F1 tanto en entrenamiento como en prueba, siendo la diferencia entre ambas curvas la medida del sobreajuste.

profundidad 2:	4 hojas	F1 entrenamiento 0.5443	F1 prueba 0.5828	brecha -0.0385
profundidad 3:	8 hojas	F1 entrenamiento 0.6298	F1 prueba 0.6463	brecha -0.0165
profundidad 5:	26 hojas	F1 entrenamiento 0.6359	F1 prueba 0.6571	brecha -0.0212
profundidad 8:	115 hojas	F1 entrenamiento 0.6602	F1 prueba 0.6065	brecha +0.0537
profundidad 10:	203 hojas	F1 entrenamiento 0.7059	F1 prueba 0.6134	brecha +0.0925
profundidad 15:	413 hojas	F1 entrenamiento 0.7602	F1 prueba 0.6125	brecha +0.1477
profundidad 20:	465 hojas	F1 entrenamiento 0.7716	F1 prueba 0.6051	brecha +0.1665
profundidad 30:	473 hojas	F1 entrenamiento 0.7724	F1 prueba 0.6051	brecha +0.1673

Mejor F1 en prueba: profundidad 5 con 0.6571

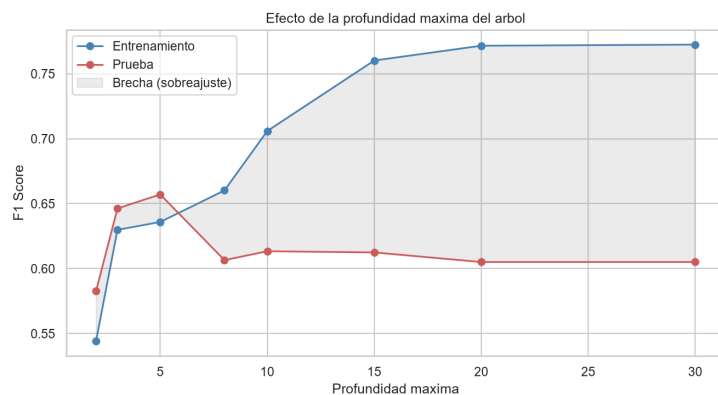


Figura 2. F1 en entrenamiento y prueba según la profundidad máxima. La zona sombreada es la brecha entre ambas.

El comportamiento observado es el que predice la teoría. Hasta la profundidad 5 ambas curvas suben juntas y la brecha es incluso negativa, sin embargo a partir de ese punto el F1 de entrenamiento continúa subiendo hasta 0.77 mientras que el de prueba cae y se estanca cerca de 0.61. Entre la profundidad 5 y la 30 el árbol pasa de 26 a 473 hojas, es decir dieciocho veces más grande, y aun así predice peor sobre datos nuevos, lo que indica que dejó de aprender patrones y comenzó a memorizar. Esto confirma además que el criterio de paro por profundidad está correctamente implementado, ya que produce exactamente el efecto esperado. El máximo en prueba se alcanza en la profundidad 5, siendo esta la configuración adoptada para el modelo final.

El segundo experimento consistió en comparar entropía contra Gini. Se entrenó el mismo árbol con los mismos datos y la misma profundidad, cambiando únicamente el criterio de impureza. Cabe aclarar que los valores de impureza de ambos criterios no son comparables entre sí ya que se encuentran en escalas distintas, entropía va de 0 a 1 y Gini de 0 a 0.5, por lo que lo que se compara son los cortes que elige cada uno y las métricas que producen.

entropia	raíz: PageValues < 0.5798	26 hojas	F1 0.6571	0.4 s
gini	raíz: PageValues < 0.5798	30 hojas	F1 0.6389	0.5 s

Métrica	Entropia	Gini	Diferencia
accuracy	0.9031	0.9015	-0.0016
precision	0.7270	0.7388	+0.0118
recall	0.5995	0.5628	-0.0366
specificity	0.9587	0.9635	+0.0048
f1	0.6571	0.6389	-0.0182

Las diferencias en las métricas son menores a dos puntos porcentuales y no resultan concluyentes, por lo que se concluye que la elección del criterio de impureza no modifica el desempeño de forma significativa en este dataset, lo cual coincide con lo reportado en la literatura. Se adoptó entropía por ser el criterio visto en clase y por presentar un F1 marginalmente mejor. La gráfica comparativa se encuentra en [resultados/comparacion_criterios.png](#).

Análisis y Conclusión

La implementación fue exitosa ya que reproduce exactamente los valores calculados a mano en el ejercicio visto en clase. El modelo entrena en menos de un segundo sobre los 9,864 registros de entrenamiento y obtiene sobre datos que nunca vio un accuracy de 0.9031 y un F1 de 0.6571, superando el piso trivial de 0.845.

La primera pregunta que hace el modelo es $\text{PageValues} < 0.5798$. Dado que la mayoría de las sesiones tienen PageValues en cero, ese corte equivale a preguntar si el usuario llegó a visitar alguna página con valor asignado, y con esa sola pregunta se separa el 78% del conjunto de entrenamiento en un grupo donde casi nadie compra. Esto coincide con la matriz de correlación, donde PageValues es la variable más relacionada con la compra con un valor de 0.49.

El principal problema del modelo es el recall de 0.5995, es decir que se le escapan cuatro de cada diez compradores. Esto se explica por el desbalance de las clases, ya que el modelo contó con 10,422 ejemplos de sesiones sin compra y solamente 1,908 con compra, por lo que aprendió mucho mejor a reconocer el patrón mayoritario. El experimento de profundidad mostró además que agrandar el árbol no resuelve este problema, sino que lo empeora: a partir de la profundidad 5 el modelo empieza a memorizar en lugar de generalizar.

Como trabajo futuro quedan tres caminos para mejorar el desempeño: ponderar las clases dentro del cálculo de la impureza para compensar el desbalance, implementar cortes categóricos que permitan recuperar las cinco variables que se descartaron, y extender el árbol a un random forest para reducir la varianza observada en el experimento de profundidad.

Referencias

Sakar, C. & Kastro, Y. (2018). Online Shoppers Purchasing Intention Dataset. UCI Machine Learning Repository. <https://doi.org/10.24432/C5F88Q>

Ramírez Uresti, J. A. sf. Tema 7: Aprendizaje Supervisado. Material del curso Inteligencia Artificial Avanzada para la Ciencia de Datos, Módulo 2. Instituto Tecnológico de Estudios Superiores de Monterrey.

Código fuente, gráficas y datos: https://github.com/A01801224/Retro1_Mod2